Atelier 1 - Bilan

Sommaire :

Page 2 : Contexte

Page 3 : Mission 1 : Nettoyer le code existant et ajouter une fonctionnalité

Page 8 : Mission 2 – Coder la partie back-office

Page 24 : Mission 3 - Tester et documenter

Page 25 : Déployer le site et gérer le déploiement continu

Contexte

Infotech services 86

Créée en 2002, la société InfoTech Services 86 n’a cessé d’enrichir son équipe en nouvelles compétences et expertises afin de proposer à ses clients TPE et PME des solutions toujours plus innovantes. ITS 86 est avant tout une équipe composée de 32 collaborateurs, administratifs, ingénieurs et techniciens. Ils sont animés par les mêmes valeurs humaines d'honnêteté, de transparence et de convivialité. Ces fondamentaux nous permettent de créer une relation de confiance, gage d'une relation saine et durable. Les activités d’ITS 86 sont organisées autour de deux pôles : **Développement et Systèmes et réseau**.

La principale activité du **pôle Développement** consiste à proposer des solutions d'hébergements sur des serveurs dédiés. Les développeurs possèdent également une expertise dans l’intégration de services, le développement logiciel et la gestion/création de bases de données.
L’activité du ***pôle Systèmes et réseau*** est répartie en deux volets : **l’Ingénierie système et réseau** (conseil & Audit, intégration, virtualisation, solutions et fourniture de matériels informatiques) et la mise en place d’un **centre de Support et d'Assistance** pour ses clients.

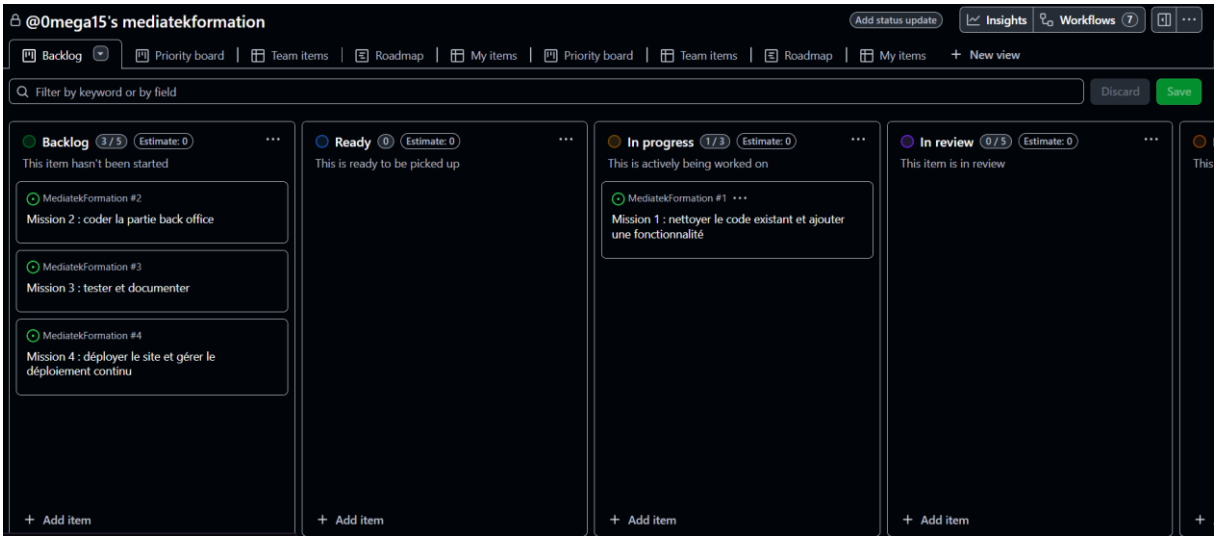
L’atelier à été faite en utilisant le logiciel Apache Netbeans en php et utilise une BDD

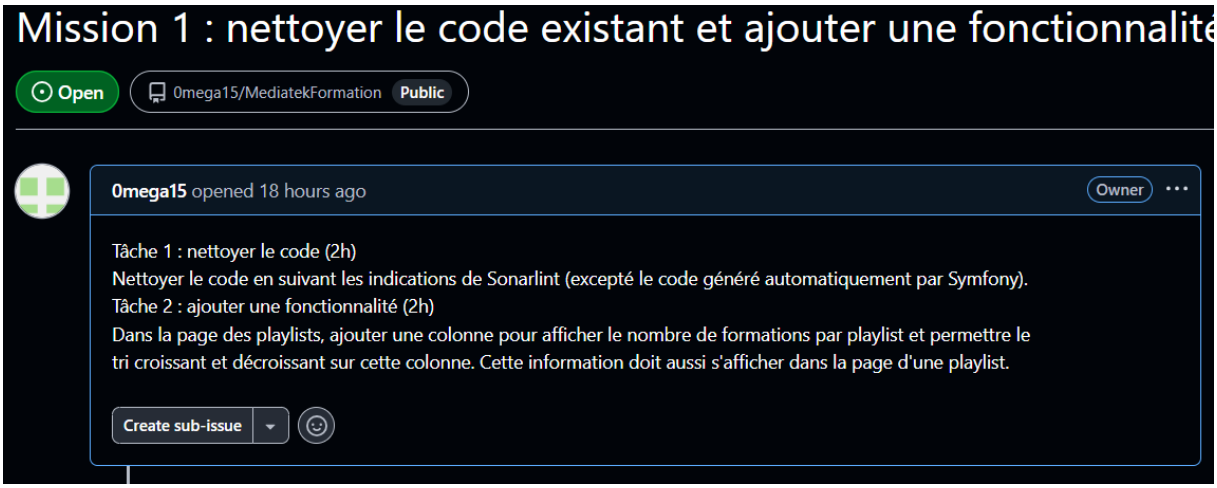
Mission 1 : Nettoyer le code existant et ajouter une fonctionnalité

Tâche 1 : Nettoyage du code

Objectif : Nettoyer le code en suivant les indications de Sonarlint afin de régler les problèmes de chaînes « en dur », et des règles de codages comme les constantes à mettre en majuscule.

Capture d'écran du kanban :





Web:BoldAndItalicTagsCheck : "" and "" tags should be used (5)

```
<p><small><i>
    Consultez nos <a class="link-secondary" href="{{ path('cgu') }}">Conditions Générales d'Utilisation</a>
</i></small></p>
```

```
<p><small><em><strong>
    Consultez nos <a class="link-secondary" href="{{ path('cgu') }}">Conditions Générales d'Utilisation</a>
</strong></em></small></p>
```

Ajout des balises pour mettre en italique et pour mettre en gras.

Web:ImgWithoutAltCheck : Image, area and button with image tags should have an "alt" attribute (4)

```
<img src="{{ formation.picture }}"
     class="card-img-top" style="width:100%;height:auto;">
```

```
<img src="{{ formation.picture }}"
     class="card-img-top" style="width:100%;height:auto;"
     alt="Logo de la formation MediaTek86">
```

Ajout d'un attribut alt pour avoir un descriptif de l'image dans le cas ou l'image ne s'afficherais pas.

Web:TableWithoutCaptionCheck : "<table>" tags should have a description (3)

```
<table class="table">
```

```
<table class="table">
    <caption>
        Les 2 dernières formations
    </caption>
```

Ajout d'une description pour chaque tableau.

---

php:S1131 : Lines should not end with trailing whitespaces (22)

```
return $this->render("pages/cgu.html.twig");
```

```
return $this->render("pages/cgu.html.twig");
```

Suppression des espaces en fin de ligne.

---

php:S1301 : "switch" statements should have at least 3 "case" clauses (1)

```
public function sort($champ, $ordre): Response{
    switch($champ){
        case "name":
            $playlists = $this->playlistRepository->findAllOrderByName($ordre);
            break;
    }
public function sort($champ, $ordre): Response
{
    if ($champ === "name") {
            $playlists = $this->playlistRepository->findAllOrderByName($ordre);
    }
    $categories = $this->categorieRepository->findAll();
    return $this->render(self::CHEMINPLAYLIST, [
        'playlists' => $playlists,
        'categories' => $categories
    ]);
}
```

Suppression du switch inutile et remplacement par une simple condition.

---

php:S1808 : Source code should comply with formatting standards (20)

```
public function index(): Response{
    $formations = $this->repository->findAllLasted(2);
    return $this->render("pages/accueil.html.twig", [
        'formations' => $formations
    ]);
}
public function index(): Response
{
    $formations = $this->repository->findAllLasted(2);
    return $this->render("pages/accueil.html.twig", [
        'formations' => $formations
    ]);
}
```

Ajout de saut à la ligne afin que les accolades entourent correctement les blocs de code.

---

php:S115 : Constant names should comply with a naming convention (1)

```
private const cheminImage = "https://i.ytimg.com/vi/";
private const CHEMINIMAGE = "https://i.ytimg.com/vi/";
```

Mise des noms des constantes en majuscules

---

php:S121 : Control structures should use curly braces (1)

php:S3973 : A conditionally executed single line should be denoted by indentation (1)

```php
public function getCategoriesPlaylist() : Collection
{
    $categories = new ArrayCollection();
    foreach($this->formations as $formation){
        $categoriesFormation = $formation->getCategories();
        foreach($categoriesFormation as $categorieFormation)
        if(!$categories->contains($categorieFormation->getName())){
            $categories[] = $categorieFormation->getName();
        }
    }
    return $categories;
}
```

```php
public function getCategoriesPlaylist() : Collection
{
    $categories = new ArrayCollection();

    foreach ($this->formations as $formation) {
        $categoriesFormation = $formation->getCategories();

        foreach ($categoriesFormation as $categorieFormation) {
            if (!$categories->contains($categorieFormation->getName())) {
                $categories[] = $categorieFormation->getName();
            }
        }
    }
    return $categories;
}
```

Mise entre accolade de la condition if et condition

---

php:S1066 : Collapsible "if" statements should be merged (1)

```php
public function removeFormation(Formation $formation): static
{
    if ($this->formations->removeElement($formation)) {
        // set the owning side to null (unless already changed)
        if ($formation->getPlaylist() === $this) {
            $formation->setPlaylist(null);
        }
    }

    return $this;
}
```

```php
public function removeFormation(Formation $formation): static
{
    if ($this->formations->removeElement($formation) && $formation->getPlaylist() === $this) {
        // set the owning side to null (unless already changed)
            $formation->setPlaylist(null);
    }

    return $this;
}
```

Fusion des deux conditions if.

Tâche 2 : ajouter une fonctionnalité

Objectif : Ajout d'une colonne dans la page des playlists afin d'y afficher le nombre de formation contenu dans cette playlist, de plus la colonne est filtrable par nombre de formation. L'information a aussi été ajouté dans la page d'une playlist.

Temps estimé 2h, temps réel 2h30.

Page des playlists contenant la nouvelle colonne.

```
38  <th class="text-left align-top" scope="col">
39      nombre de formation<br />
40      <a href="{{ path('playlists.sort', {champ:'nombre_formation', ordre:'ASC'}) }}" class="btn btn-info btn-sm active" role="button" aria-pressed="
41      <a href="{{ path('playlists.sort', {champ:'nombre_formation', ordre:'DESC'}) }}" class="btn btn-info btn-sm active" role="button" aria-pressed
42  </th>
```

Ajout de la nouvelle colonne ainsi que des boutons permettant le tri en fonction du nombre de formation. Appel de la fonction sort et envoi en paramètre la colonne à trier et l'ordre (croissant ou décroissant).

```
        </thead>
48      <tbody>
49          <!-- boucle sur les playlists -->
50          {% if playlists|length > 0 %}
51              {% for k in 0..playlists|length-1 %}
52                  <tr class="align-middle">
53                      <td>
54                          <h5 class="text-info">
55                              {{ playlists[k].name }}
56                          </h5>
57                      </td>
58                      <td class="text-left">
59                          {% set categories = playlists[k].categoriesplaylist %}
60                          {% if categories|length > 0  %}
61                              {% for c in 0..categories|length-1 %}
62                                   {{ categories[c] }}
63                              {% endfor %}
64                          {% endif %}
65                      </td>
66                      <td class="text-left">
67                          {{ playlists[k].formations|length }}
68                      </td>
69                      <td class="text-center">
70                          <a href="{{ path('playlists.showone', {id:playlists[k].id}) }}" class="btn btn-secondary">Voir détail</a>
71                      </td>
72                  </tr>
73              {% endfor %}
74          {% endif %}
```

Dans la boucle permettant d'ajouter les informations des playlists, dans la nouvelle colonne : pour chaque playlist récupère le nombre de formation contenue dedans.

```php
#[Route('/playlists/tri/{champ}/{ordre}', name: 'playlists.sort')]
public function sort($champ, $ordre): Response
{
    if ($champ === "name") {
        $playlists = $this->playlistRepository->findAllOrderByName($ordre);
    }
    if ($champ === "nombre_formation") {
        $playlists = $this->playlistRepository->findAllOrderByFormationNumber($ordre);
    }
    $categories = $this->categorieRepository->findAll();
    return $this->render(self::CHEMINPLAYLIST, [
        'playlists' => $playlists,
        'categories' => $categories
    ]);
}
```

Ajout dans la fonction sort d'une condition qui appelle la fonction « findAllOrderByFormationNumber » si le champ est « nombre_formation » qui prend en paramètre l'ordre de tri.

```php
public function findAllOrderByFormationNumber($ordre): array
{
    return $this->createQueryBuilder('p')
        ->select('p, COUNT(f.id) AS HIDDEN formations_count')
        ->leftJoin('p.formations', 'f')
        ->groupBy('p.id')
        ->orderBy('formations_count', $ordre)
        ->getQuery()
        ->getResult();
}
```

Ajout de la fonction « findAllOrderByFormationNumber » qui prend en paramètre l'ordre du tri. La requête crée une colonne caché « formations_count » qui pour chaque playlist p contiendra le nombre de formation qui lui sont associé puis la colonne est trié en fonction de l'ordre.

```html
<strong>nombre de formation : </strong>
{{ playlistformations|length }}
```

Dans la page d'une playlist choisi affiche le nombre de formation qu'elle contient.

Mission 2 – Coder la partie back-office

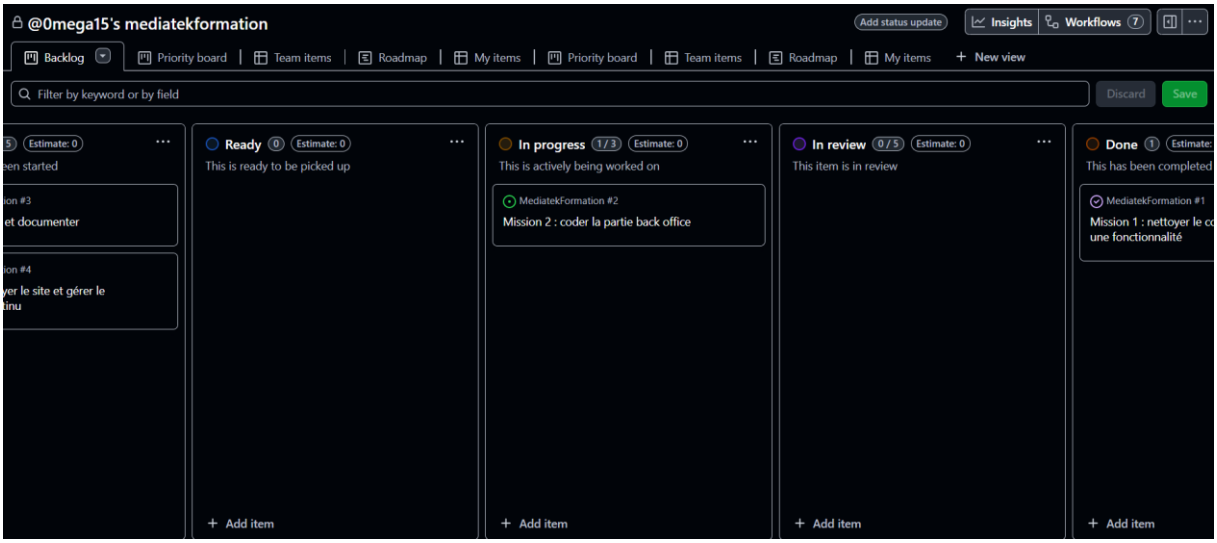
Tâche 1 : gérer les formations

Objectif : Ajout d'une partie back-office destiné aux admin contenant pour le moment, l'affichage des formations qui peut être trié comme dans le front-office ainsi que de 2 boutons éditer et supprimer pour chaque formation. L'appui sur le bouton suppression supprime la formation correspondante. L'appui sur le bouton édit ouvre une nouvelle page qui contient un formulaire pré-rempli avec les informations de la formation choisi. Un autre bouton en haut de la page permet l'ajout d'une formation, ce bouton renvoi sur

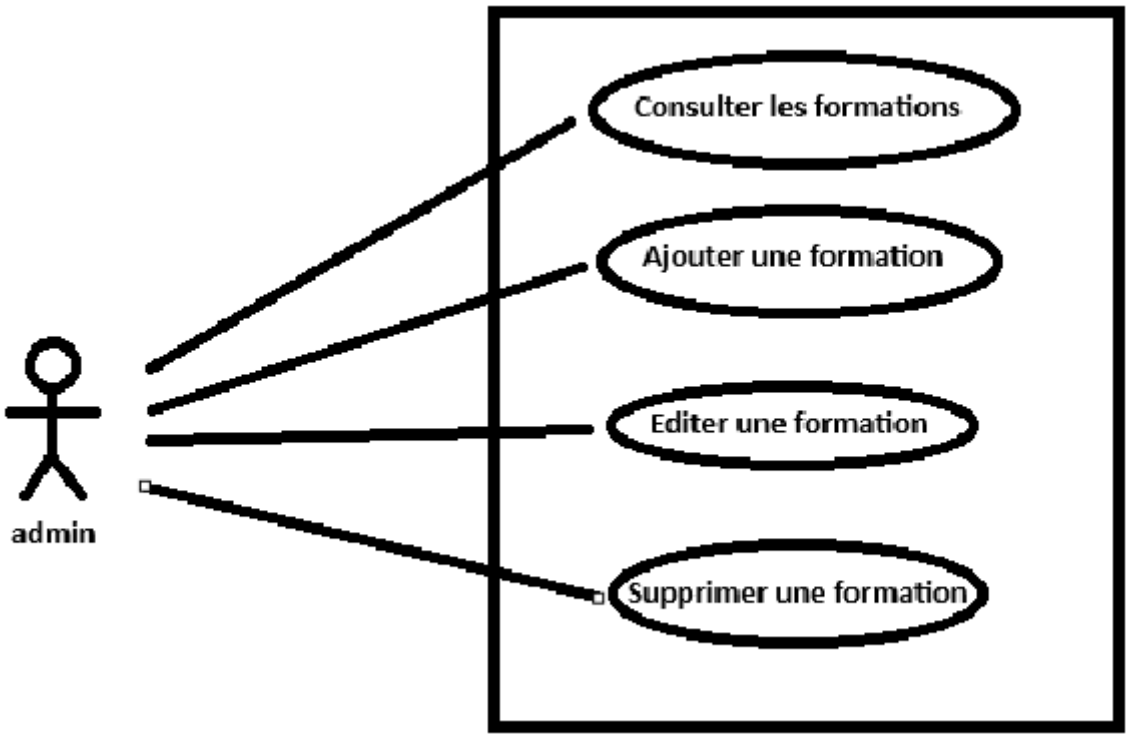
une nouvelle page identique à la page d'édition mais les champs du formulaire sont vides.

Temps estimé 5h, temps réel 6h

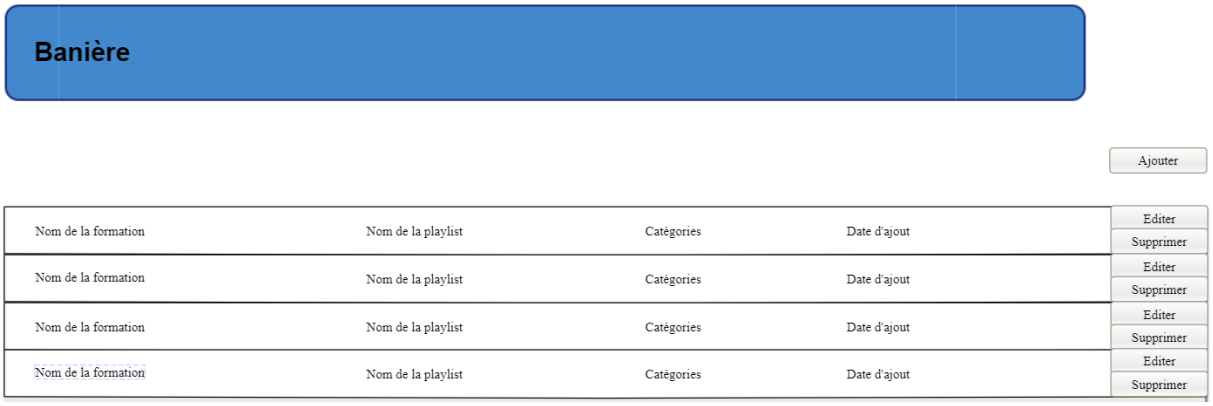
Capture d'écran du kanban



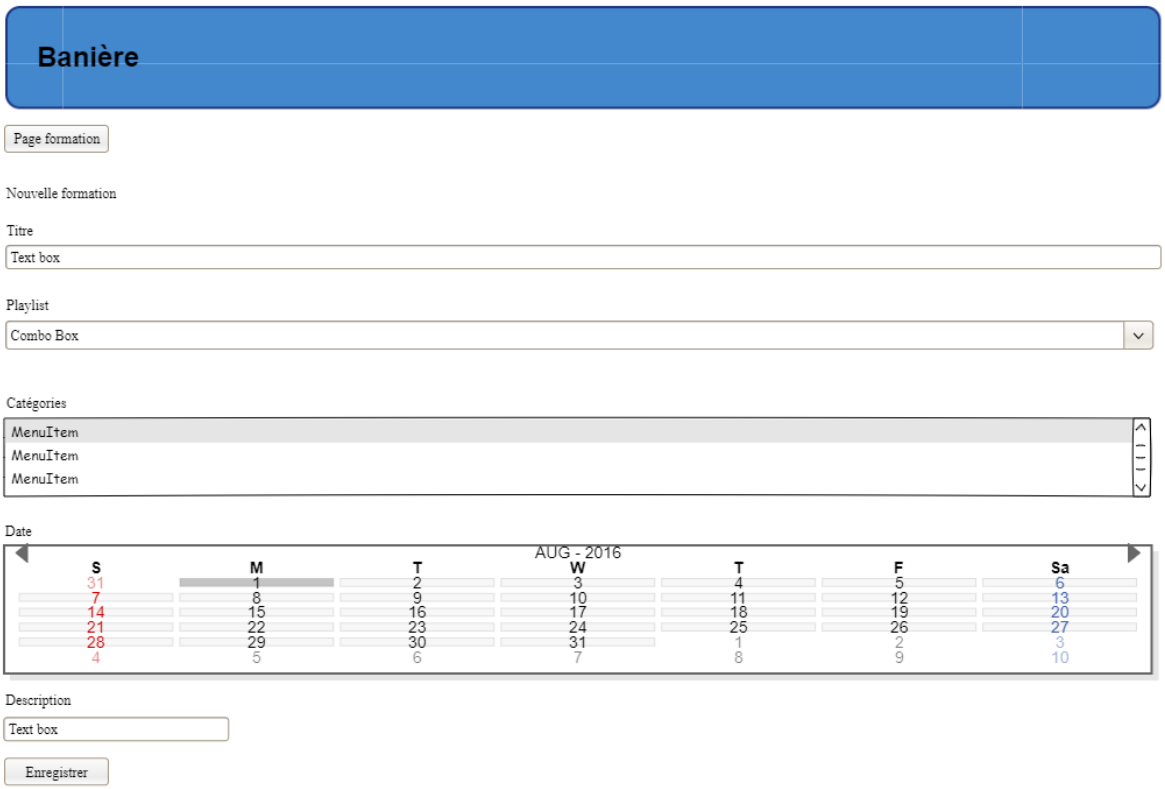
Diagrame d’utilisation concernant cette tâche uniquement



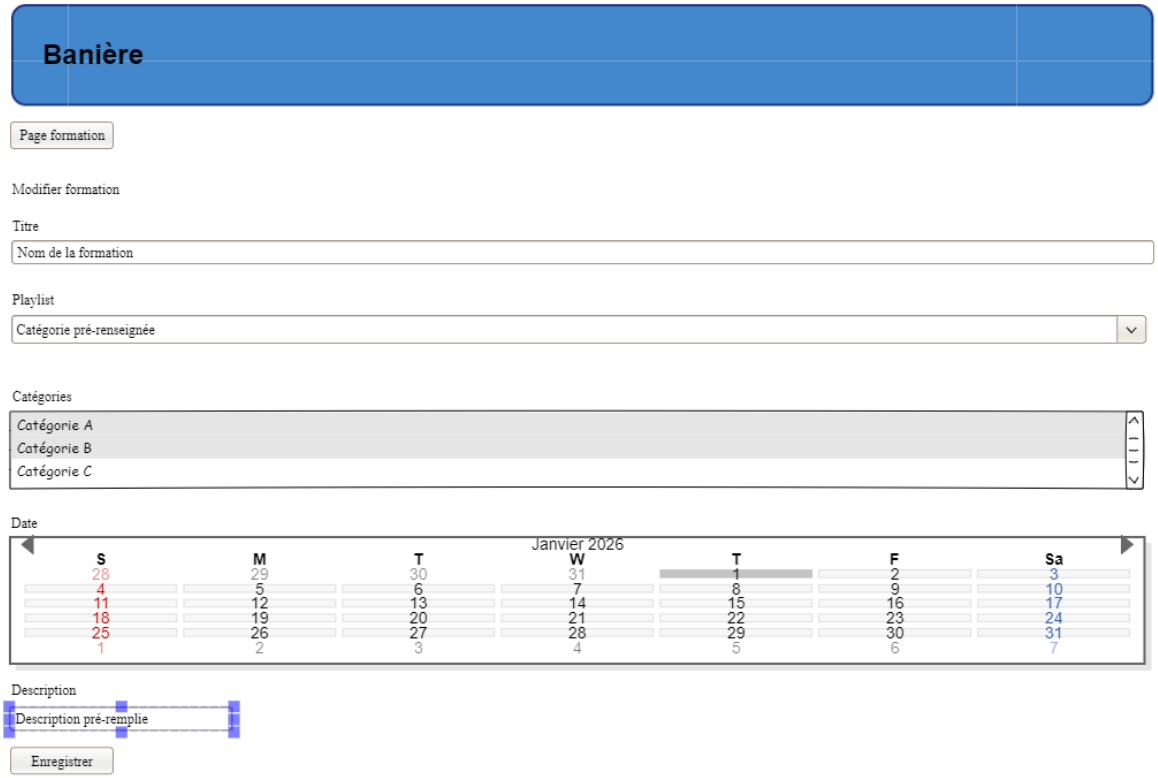
Maquette de la page listant les formations :



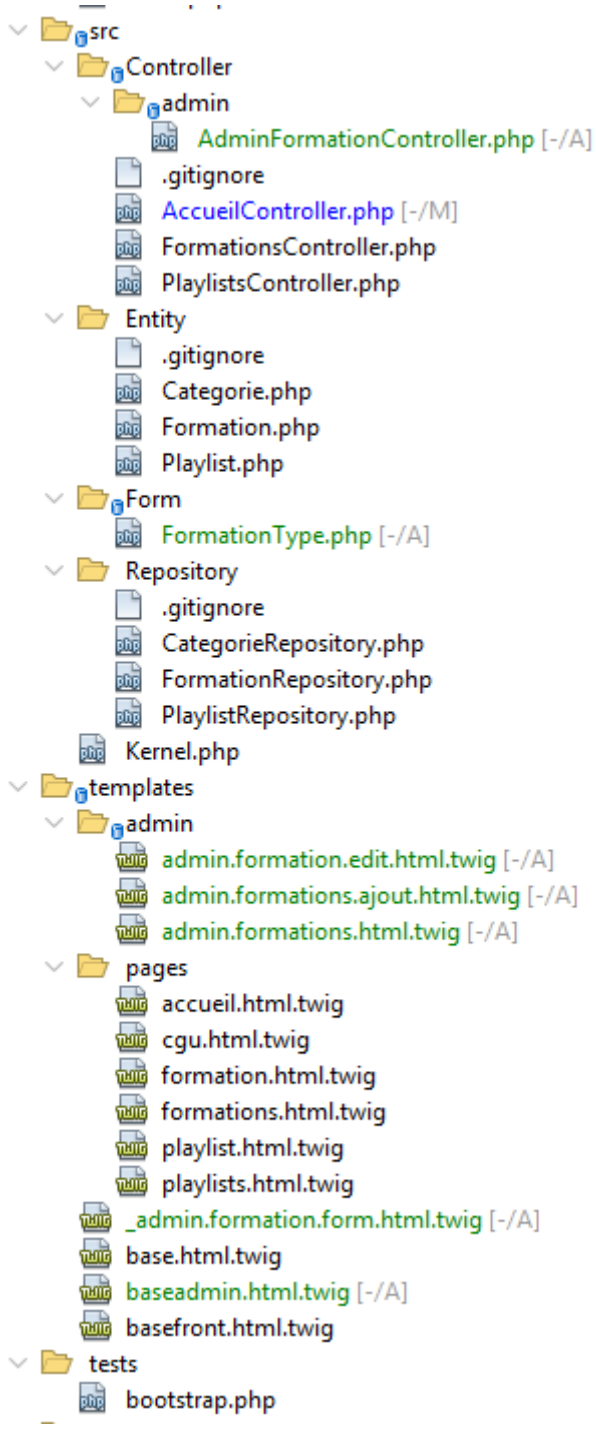
Maquète de la page permettant d’ajouter une formation :



Maquette de la page de modification :



Arborescence des fichiers, les fichiers ajouté sont en vert.



Baseadmin.html.twig permet d'avoir une base pour les autres pages admin, à savoir la banières et les boutons permettant de changer de page.

Admin.formations.html.twig permet d'afficher toutes les formations disponibles ainsi que les boutons de filtrage, d'ajout, de modification et de suppression.

_admin.formation.form.html.twig permet d'afficher les différentes lignes du formulaire permettant de modifier ou ajouter une formation.

Admin.formation.edit.html.twig et admin.formations.ajout.html.twig récupère tout 2 le contenu de admin.formation mais modifie le titre de la page.

FormationType permet de créer le formulaire et ses éléments.

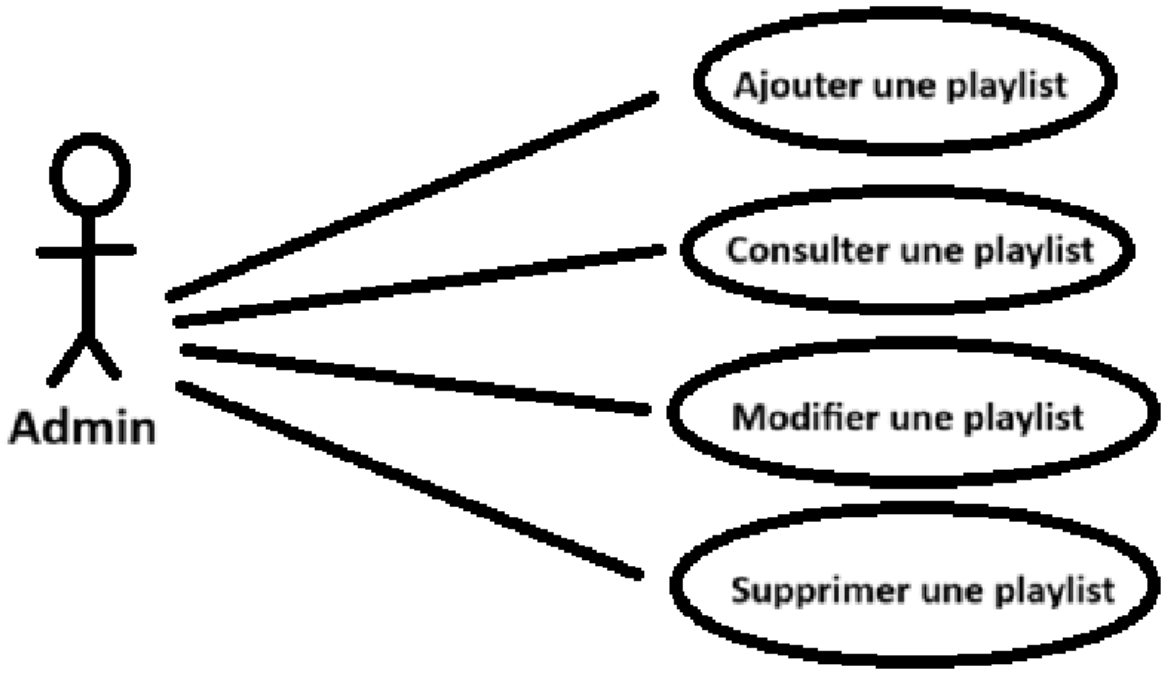
AdminFormationController.php contient les fonctions de tri, recherche, ajout et de modification des formations.

Tâche 2 : gérer les playlists

Objectif : Ajout d'une dans la partie back-office une page destiné a la gestion des playlists qui contient entre autres : la liste des playlists avec pour chacune un bouton supprimer qui affiche un message si la playlist contient des formations et un bouton éditer qui permet de modifier la playlist. De plus les mêmes tris présents sur les colonnes front-office, sont présent coté back-office. Ajout d'une page qui permet d'ajouter ou modifier une playlist.

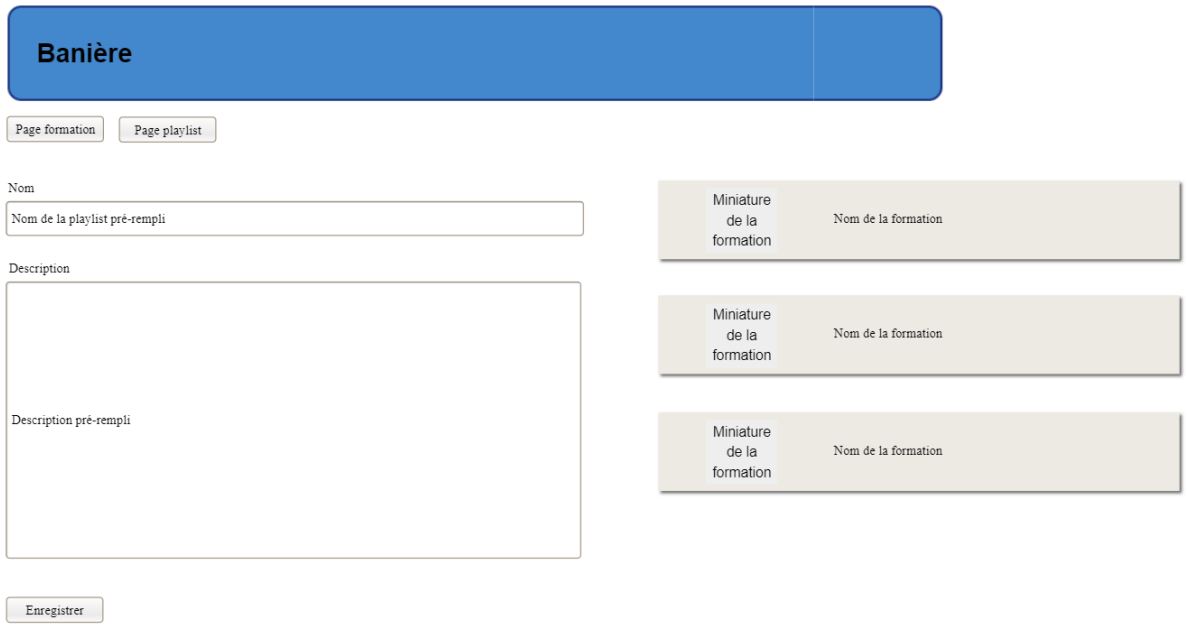
Temps estimé 5h, temps réel 4h

Diagramme d'utilisation de cette tâche :

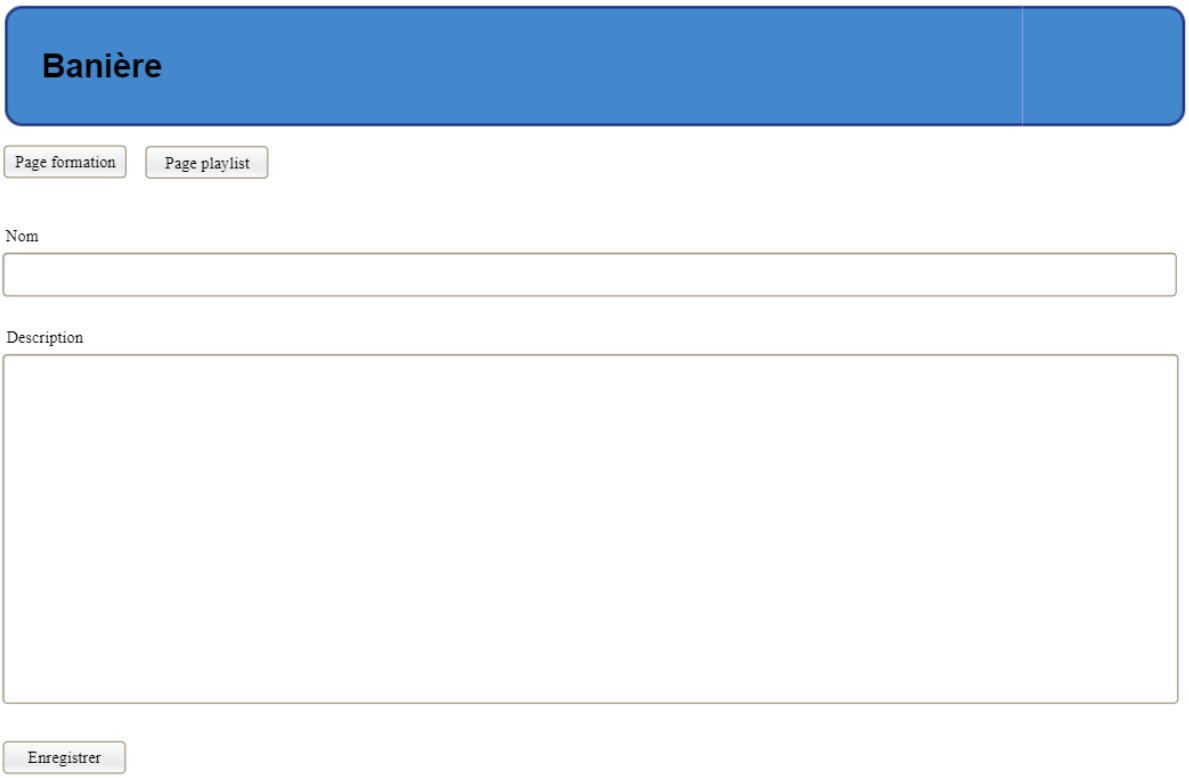


Maquette de la page des playlists :

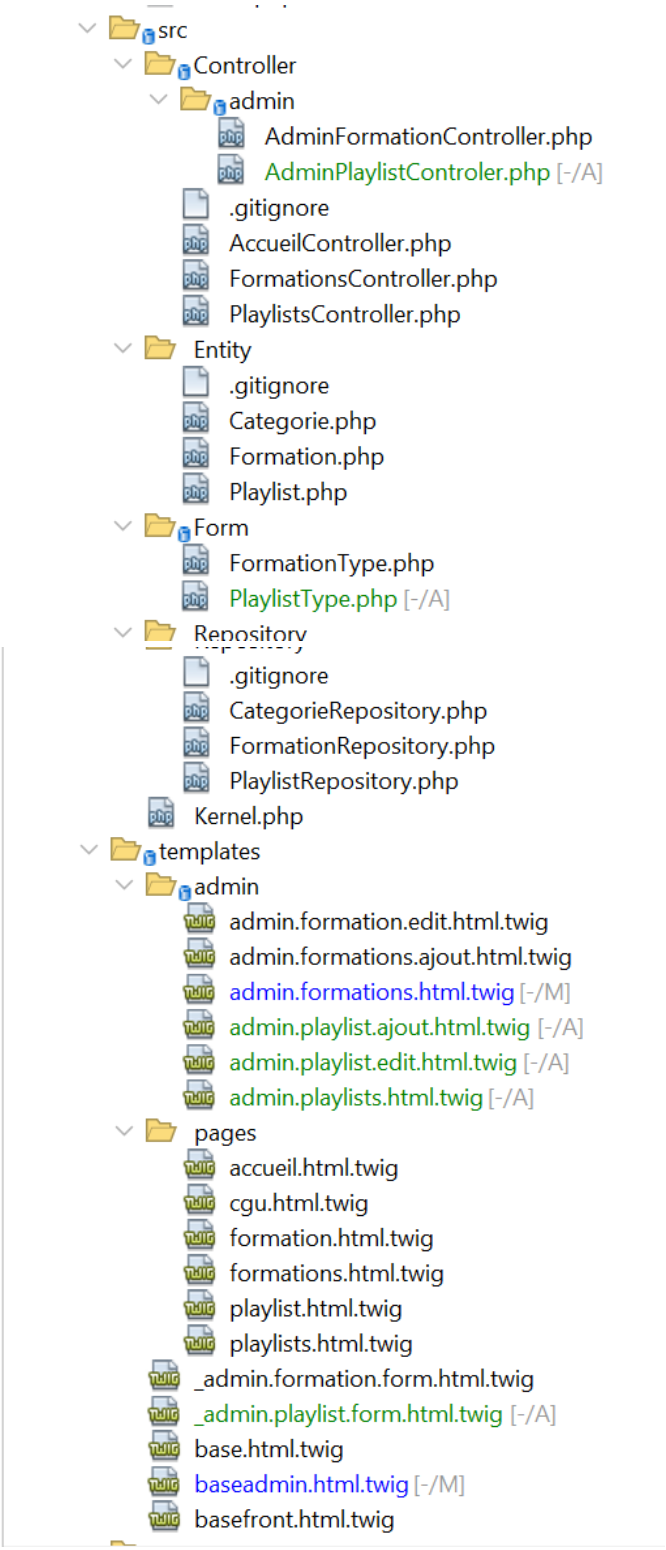
Maquette de la page de modification d'une playlist :



Maquette de la page d’ajout d’une playlist :



Arborescence des fichiers, les fichiers ajouté sont en vert.



Admin.playlists.html.twig permet d'afficher toutes les playlists disponibles ainsi que les boutons de filtrage, d'ajout, de modification et de suppression.
_admin.playlist.form.html.twig permet d'afficher les différentes lignes du formulaire permettant de modifier ou ajouter une playlist.

Admin.playlist.edit.html.twig et admin. playlist.ajout.html.twig récupère tout 2 le contenu de admin.formation mais modifie le titre de la page.
Admin.playlist.edit.html.twig affiche en plus la liste des formations qui lui sont rattaché.

PlaylistType permet de créer le formulaire et ses éléments.

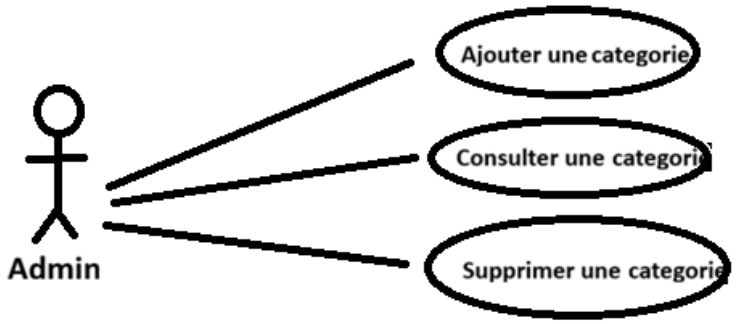
AdminPlaylistController.php contient les fonctions de tri, recherche, ajout et de modification des playlists.

Tâche 3 : gérer les catégories

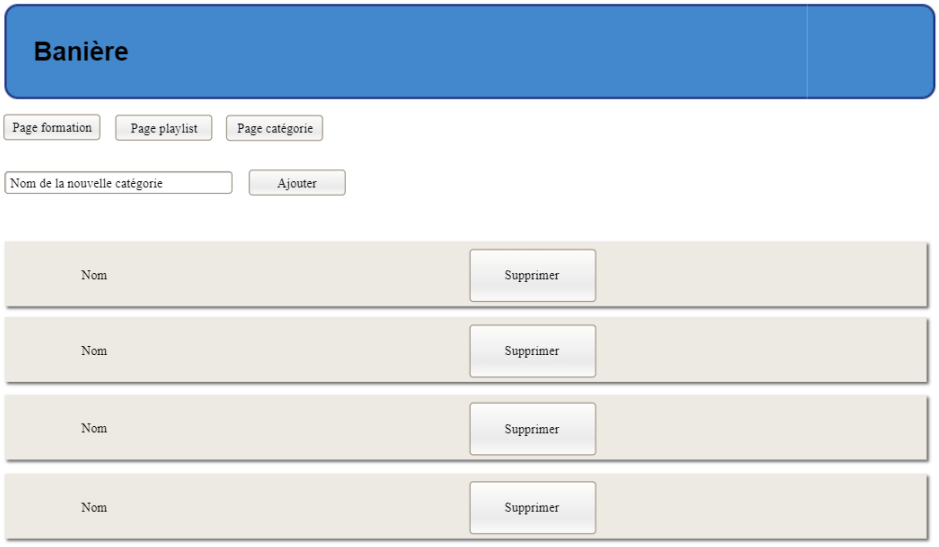
Objectif : Ajout d'une dans la partie back-office une page destiné à la gestion des catégories qui contient entre autres : la liste des playlists avec pour chacune un bouton supprimer qui affiche un message si la catégorie est présente dans une formation. Un bouton et un champ de texte permet d'ajouter une catégorie.

Temps estimé : 3h, temps réel : 2h

Diagramme d'utilisation de cette tâche :

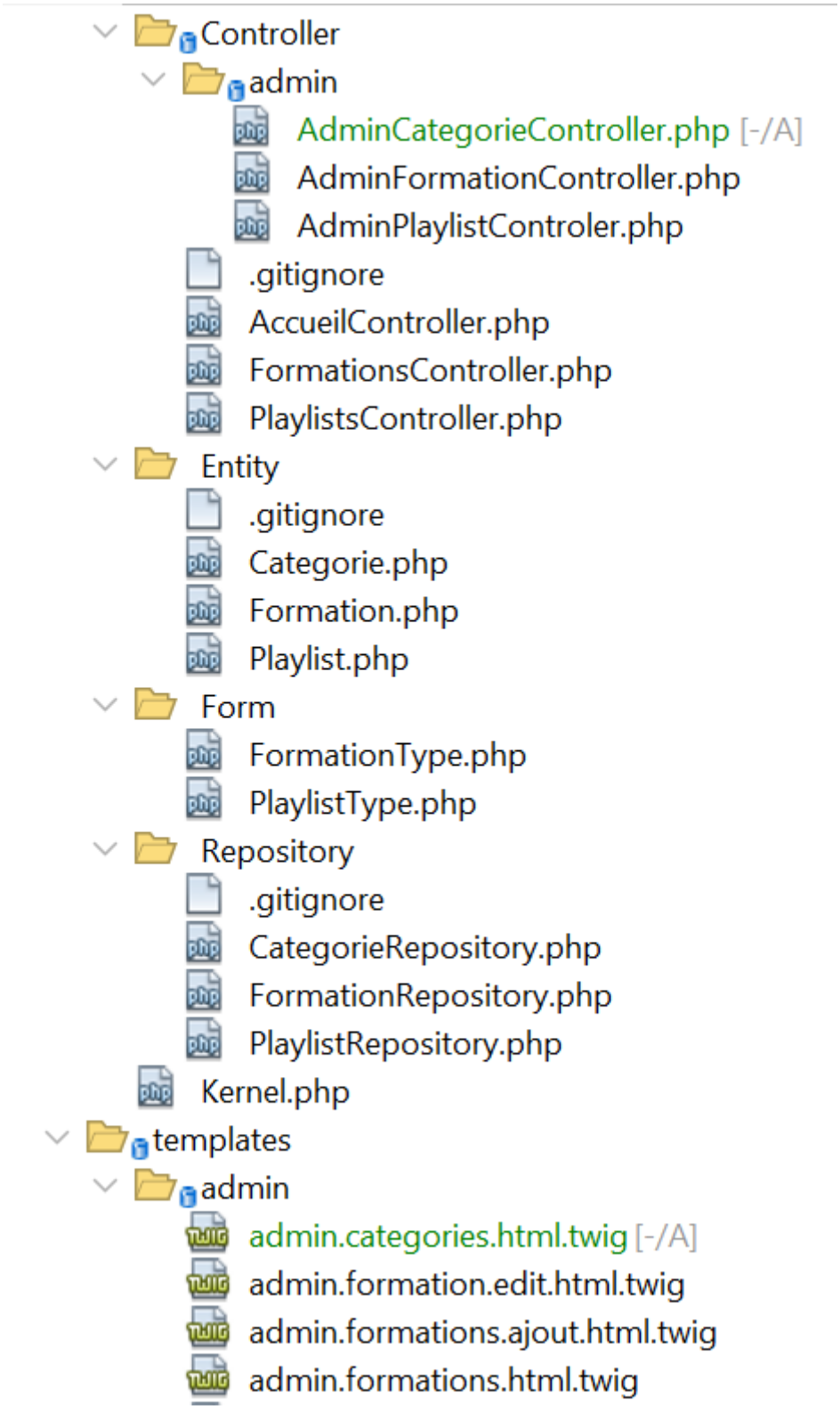


Maquette de la page des catégories :



:

Arborescence des fichiers, les fichiers ajouté sont en vert.



Admin.categories permet d'afficher les élément visuels à savoir les boutons pour ajouter ou supprimer les categories, et d'afficher la liste des categories.

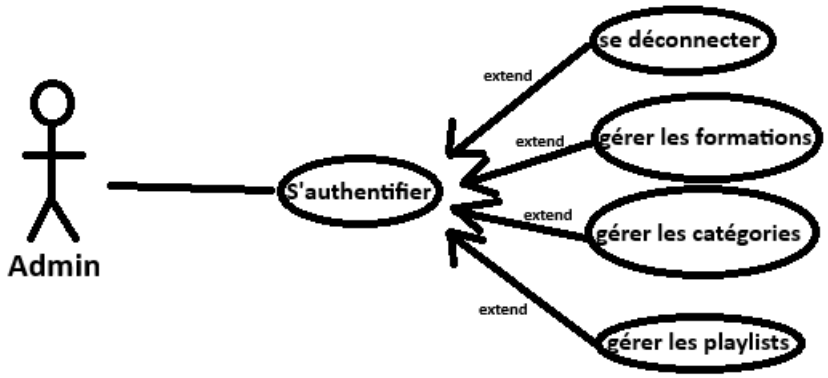
AdminCategorieController contient les fonctions pour ajouter et supprimer les categories.

Tâche 4 : ajouter l'accès avec authentification

Objectif : Ajout d'une authentification pour accéder au back office ainsi qu'un lien de déconnexion si l'admin est connecté.

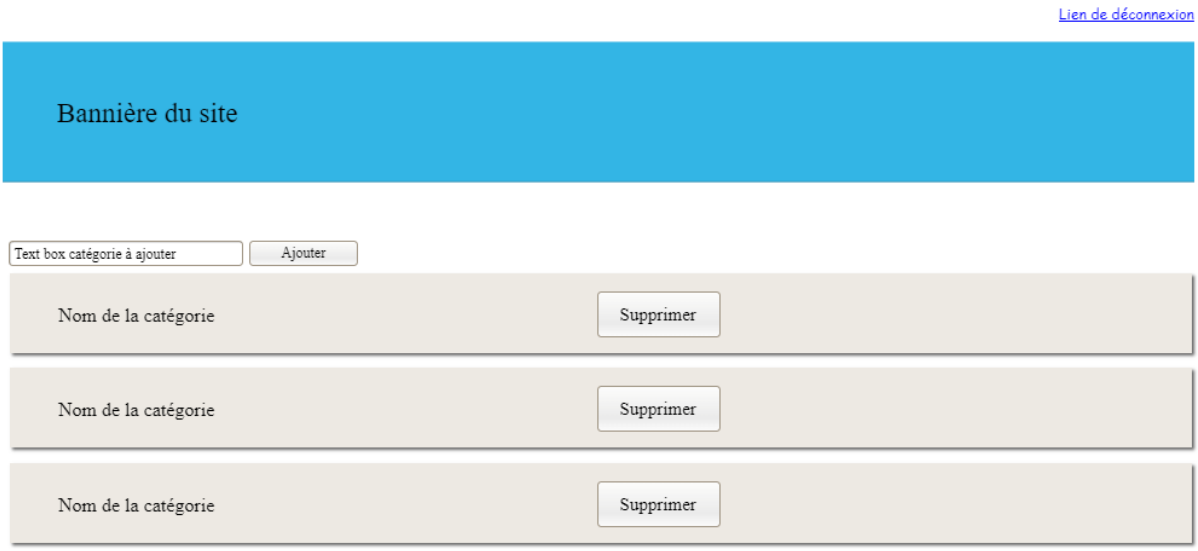
Temps estimé 4h, temps réel 3h.

Diagramme d'utilisation de cette tâche :



Maquette de la page de connexion :

Maquette d'une page contenant le lien de déconnexion :



Modification de base.html.twig :

```
<body>
    <!-- entête -->
    <div class="container text-end">
        {% if app.user %}
            <a href="{{ path('logout') }}">se déconnecter</a>
        {% endif %}
    </div>
    {% block top %}{% endblock %}
```

Ajout d'un lien dans « se déconnecter » dans l'entête si un utilisateur est connecté donc un admin. Si aucun utilisateur n'est connecté le lien ne s'affiche pas.

Ajout de User.php :

```php
<?php

namespace App\Entity;

use App\Repository\UserRepository;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface;
use Symfony\Component\Security\Core\User\UserInterface;

#[ORM\Entity(repositoryClass: UserRepository::class)]
#[ORM\UniqueConstraint(name: 'UNIQ_IDENTIFIER_USERNAME', fields: ['username'])]
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 180)]
    private ?string $username = null;

    /**
     * @var list<string> The user roles
     */
    #[ORM\Column]
    private array $roles = [];

    /**
     * @var string The hashed password
     */
    #[ORM\Column]
    private ?string $password = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getUsername(): ?string
    {
        return $this->username;
    }
```

Classe User permettant de récupérer les informations de la table « user » à savoir les getters et setters des roles, des mots de passe et des noms d'utilisateurs.

Ajout de UserRepository :

```php
class UserRepository extends ServiceEntityRepository implements PasswordUpgraderInterface
{
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, User::class);
    }

    /**
     * Used to upgrade (rehash) the user's password automatically over time.
     */
    public function upgradePassword(PasswordAuthenticatedUserInterface $user, string $newHashedPassword): void
    {
        if (!$user instanceof User) {
            throw new UnsupportedUserException(sprintf('Instances of "%s" are not supported.', $user::class));
        }

        $user->setPassword($newHashedPassword);
        $this->getEntityManager()->persist($user);
        $this->getEntityManager()->flush();
    }
```

Permet de modifier le hashage du mot de passe automatiquement afin d'éviter l'obsolecense du hashage.

Ajout de UserFixture :

```php
class UserFixture extends Fixture
{

    private $passwordHasher;

    public function __construct(UserPasswordHasherInterface $passwordHasher)
    {
        $this->passwordHasher = $passwordHasher;
    }
    public function load(ObjectManager $manager): void
    {
        $user = new User();
        $user->setUsername("admin");
        $plaintextPassword = "admin";
        $hashedPassword = $this->passwordHasher->hashPassword(
                $user,
                $plaintextPassword
        );
        $user->setPassword($hashedPassword);
        $user->setRoles(['ROLE_ADMIN']);
        $manager->persist($user);
        $manager->flush();
    }
}
```

Permet d'ajouter dans la bdd un nouvel utilisateur dont le nom est admin, le mot de passe est admin qui est ensuite hasher et le role est ROLE_ADMIN.

Ajout de index.html.twig :

```
{% extends 'base.html.twig' %}

{% block body %}
    {% if error %}
        <div>{{ error.messageKey|trans(error.messageData, 'security') }}</div>
    {% endif %}

    <form action="{{ path('app_login') }}" method="post">
        <h3>Authenifiez-vous</h3>

        <label for="username">Login:</label>
        <input type="text" id="username" name="_username" value="{{ last_username }}"
               class="form-control" required autofocus />

        <label for="password">Password:</label>
        <input type="password" id="password" name="_password"
               class="form-control" required />

        <input type="hidden" name="_csrf_token" value="{{ csrf_token('authenticate') }}">

        <br />
        <button class="btn btn-lg btn-primary" type="submit">se connecter</button>
    </form>
{% endblock %}
```

Permet d'afficher la page contenant les textbox pour les nom d'utilisateur et le mot de passe. Un champ caché permet d'empecher les attaques CSRF.

Ajout de LoginController :

```
class LoginController extends AbstractController
{
    #[Route('/login', name: 'app_login')]
    public function index(AuthenticationUtils $authenticationUtils): Response
    {
        $error = $authenticationUtils->getLastAuthenticationError();

        $lastUsername = $authenticationUtils->getLastUsername();

        return $this->render('login/index.html.twig', [
            'last_username' => $lastUsername,
            'error' => $error
        ]);
    }

    #[Route('/logout', name: 'logout')]
    public function logout()
    {

    }
}
```

Permet de déconnecter l'utilisateur.

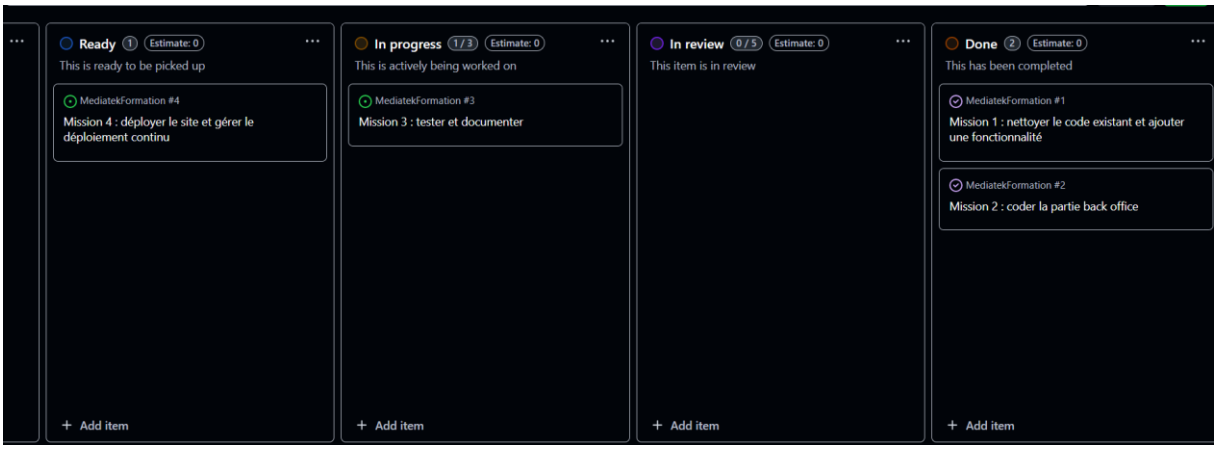
Mission 3 - Tester et documenter

Tâche 1 : Gérer les tests

Objectif : Exécuter différents tests sur les fonctions du site.

Temps estimé : 7h, temps réel :12h

Capture d'écran du kanban :

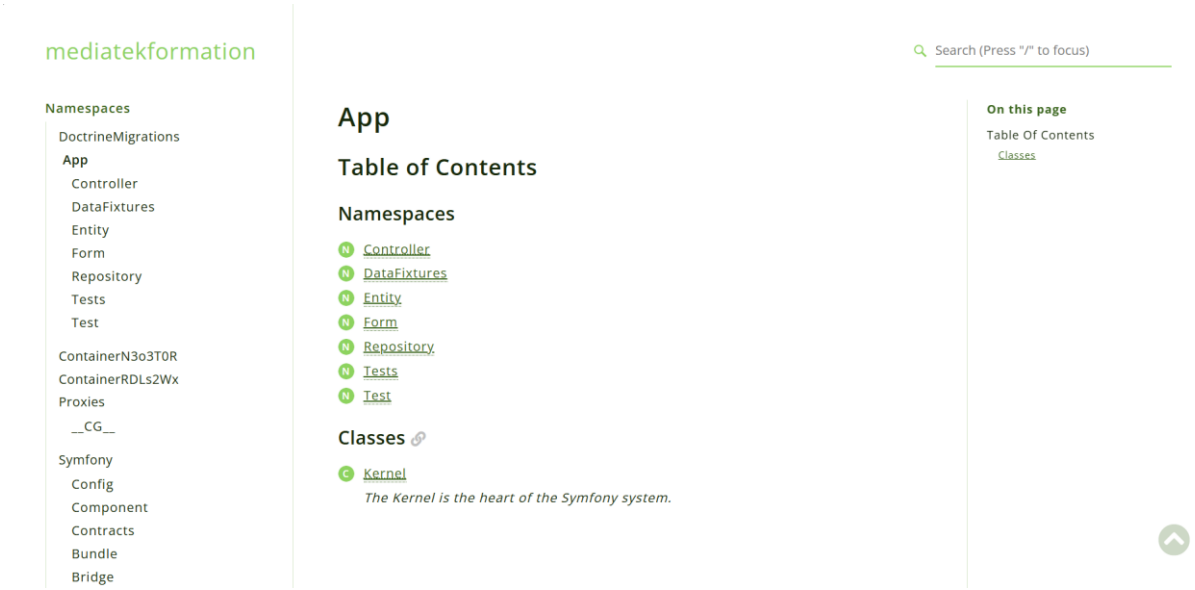


Tâche 2 - Créer la documentation technique

Objectif : Générer la documentation technique

Temps estimé 1h, temps réel 1h

Première page de la documentation technique :



Tâche 3 - Créer la documentation utilisateur

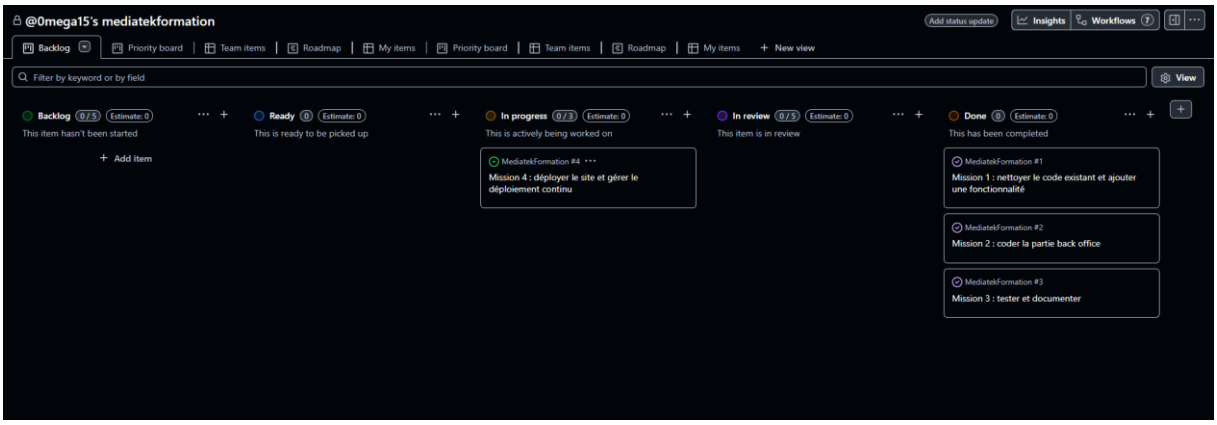
Objectif : Créer une vidéo présentant les fonctionnalités du site

Temps estimé 1h, temps réel 30min

Mission 4 : Déployer le site et gérer le déploiement continu

Tâche 1 : Déployer le site

Temps estimé 2h, temps réel 4h



Résumé des étapes du site : Création d'un compte planetHoster, import de la BDD puis import des fichiers du site, et publication.

Tâche 2 : gérer la sauvegarde et la restauration de la BDD

Temps estimé 1h, temps réel 1h

Etapes de création d'une sauvegarde régulière de la bdd : Créer un fichier de script pour enregistrement de la bdd dans un autre fichier, Mise du fichier de script au format linux puis transmission chez l'hébergeur et enfin Ajout d'une tâche Cron.

L'exécution du script n'est pas possible dans la version gratuite de l'offre de l'hébergeur.

Tâche 3 : Mettre en place le déploiement continu

Temps estimé 1h, temps réel 1h

Script :

```
on: push
name: Deploy website on push
jobs:
   web-deploy:
        name: Deploy
        runs-on: ubuntu-latest
        steps:
        - name: Get latest code
          uses: actions/checkout@v2
        - name: Sync files uses: SamKirkland/FTP-Deploy-Action@4.3.0
          with:
          server: sftp://node16-eu.n0c.com
          server-dir: /public_html/mediatekformation/
          username: bcnqzstnej
          password: ${{ secrets.ftp_password }}
```